



University of Piraeus
School of Information and Communication Technologies
Department of Digital Systems

Level: Master's Programme of Studies

Network Security

Case 2

Supervising Professor: Prof. Christos Xenakis

Ioannis Kalaitzidis

ioannis_kalaitzidis@ssl-unipi.gr

Piraeus

01/11/2025

Abstract

Low-level packet sniffing using raw sockets on Kali Linux (VMware) with the NIC in promiscuous mode. The captures are saved as pcap files and analyzed with Wireshark.

Contents

Introduction.....	
Practical part of the assignment.....	
Bibliography.....	

Introduction

In this exercise we deal with low-level packet sniffing using raw sockets. A socket, in general, is a communication mechanism used by the operating system to allow applications to exchange data over a network. Raw sockets are a special type of socket that allows an application to have direct access to network packets, without the normal protocol processing by the kernel. In other words, raw sockets bypass the kernel's standard filtering and intervene in the process of delivering packets to applications. Thus, at first, the packets are neither filtered nor separated for delivery to UDP/TCP sockets, and we can see all the bytes and headers as the network card receives them. In this way we gain access to information that we would not otherwise easily see.

The network card (NIC) is the physical device that connects the computer to the network, and each NIC has a unique MAC address. The NIC works in different modes: promiscuous mode (on wired and wireless networks), where it reads all the frames passing through the interface (i.e. the network card in the operating system), and monitor mode (on wireless networks only), where it reads frames on the specific wireless channel (i.e. the frequency on which the network card transmits/receives). For the exercise, we will enable promiscuous mode, so that the raw socket receives all the interface's traffic and we can analyse packets.

The VM sees a virtual NIC managed by the host. This virtual card is not the same as the real physical Wi-Fi card. So, even if the host is connected wirelessly, the VM does not have direct access to the Wi-Fi traffic and sees only what the host provides it, depending on the mode (NAT or Bridged).

- NAT: The VM shares the host's network connection and gets an internal IP. External traffic appears with the host's IP. In practice, the VM mainly sees its own traffic, which limits the ability to sniff the traffic of other LAN devices.
- Bridged: The VM behaves as a separate device on the local network (it gets an IP from the router) and can exchange packets directly with other LAN devices. This allows broader sniffing, but exposes the VM to the local network and entails greater risk and legal liability.

Practical part of the assignment

Introduction:

We will carry out the exercise on Kali Linux (kali-linux-2025.3-vmware-amd64) via VMware Workstation Pro (latest version), because we need root privileges for the raw sockets to work and to change the card's mode (promiscuous). For isolation and security reasons, we choose NAT as the default. With this setting the VM is not directly exposed to the local network and we mainly see the traffic that the VM itself generates/receives, without accidentally collecting the traffic of other LAN devices. If we need to analyse the traffic of other devices, then we will choose Bridged or use a physical Wi-Fi adapter in monitor mode. On the VM we will also run Wireshark in parallel for graphical analysis and the application of filters.

Starting the procedures:

From the resource characteristics available for this OS, we observe that the network adapter is on NAT. Therefore, we will not make any modification to this setting.

We observe that, either with the `ip link show` command or with the `ip -br link` command, we can check our interface. The interface we will use in the exercise is `eth0`, because it is the only one that is active (UP) and corresponds to the virtual card provided by VMware (via NAT).

The next step is to check whether promiscuous mode is enabled. With the command `ip link show eth0 | grep PROMISC` (the same is done with the command `ip link show eth0`) we can check whether this mode is enabled. If it is not, we enable it with the command `sudo ip link set eth0 promisc on` (and, correspondingly, to disable it, it is the same command but ending in `off` instead of `on`).

Once we completed the above procedures, we download the libpcap library. Then we create a packet sniffer in the C programming language using the library. We

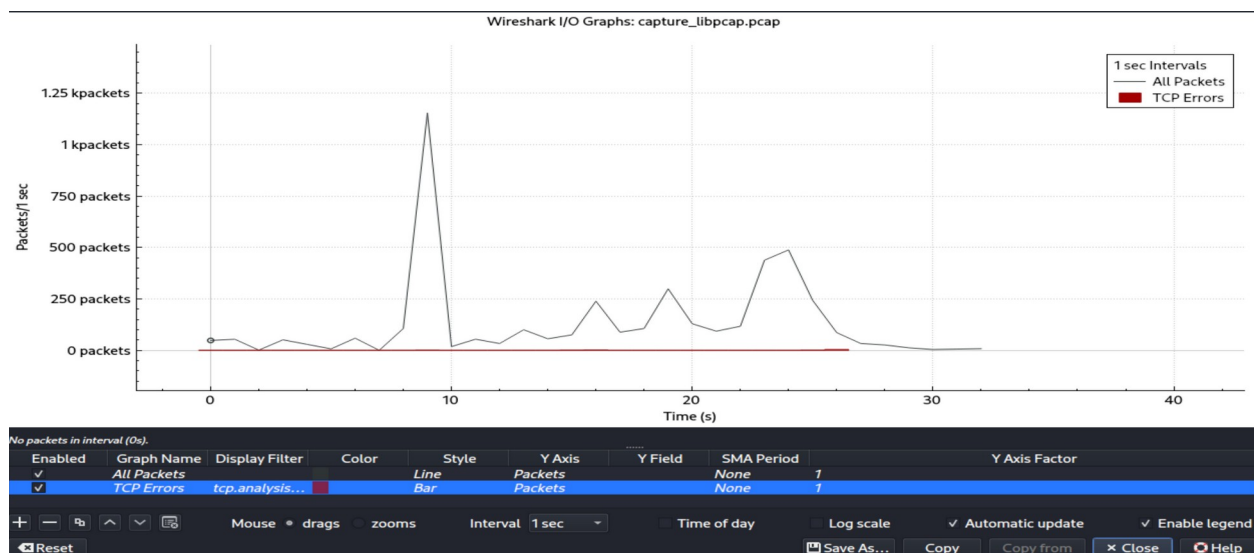
created the file `pcap_sniffer.c` in the Downloads folder, and it contains the modified code based on the one from the slide provided by the instructor.

We go to the Downloads folder, where we have the file, and we compile it. With this command, besides 'entering' the Downloads folder, we also searched for any file with a `.c` extension. The only file in this case was the sniffer, so we proceed to compile it.

After compiling the sniffer with the command `gcc pcap_sniffer.c -o pcap_sniffer -lpcap` using the library, and making sure it produces no problem during compilation, we run the sniffer file and set as interface the port that is open (`sudo ./pcap_sniffer eth0`). This starts the capture of packets, and we can stop the process by pressing `Ctrl + C` on the keyboard. Upon completing the above process, the file (`capture_libpcap.pcap`) is created, which we will open with Wireshark and which contains all the information about the packets. We open the file with the command `wireshark <file name>` (`wireshark capture_libpcap.pcap`). Wireshark will open and display all the packets.

Next, we can select Statistics to draw the first conclusions:

- From the analysis of the protocol hierarchy, it is observed that the majority of the traffic is based on IPv4, with UDP (56.4%) and TCP (42.5%) as the dominant transport protocols. The upper layers mainly include QUIC and TLS, which are used for fast and secure web connections, while DNS, ICMP and ARP appear in smaller proportions and concern auxiliary network functions.



- From the flow graph (I/O Graph), a variation is observed in the number of packets per second, with a sharp peak at around the 10th second and smaller increases afterwards. This increase indicates periods of intense network activity, probably during data loading or the start of new connections. No significant TCP errors are detected, which indicates smooth operation and stable data transfer.

Conclusion:

The exercise demonstrated the practical operation of a low-level packet sniffer and the effectiveness of Wireshark in analysing and visualising network traffic.

Bibliography

Moon, S. (2020, July 26). How to code raw sockets in C on Linux - BinaryTides. Retrieved from <https://www.binarytides.com/raw-sockets-c-code-linux/>

nospaceships/raw-socket-sniffer: Packet capture on Windows without a kernel driver. (n.d.). Retrieved from <https://github.com/nospaceships/raw-socket-sniffer>

packet(7) - Linux manual page. (n.d.). Retrieved from <https://man7.org/linux/man-pages/man7/packet.7.html>

Harrichael. (2014, July 25). C raw socket receive or sniff incoming packets only. Retrieved from <https://stackoverflow.com/questions/24964586/c-raw-socket-receive-or-sniff-incoming-packets-only>

Raw Socket programming - Efficient Packet Sniffer - Applications / Programming - Unix Linux Community. (2013, June 26). Retrieved from <https://community.unix.com/t/raw-socket-programming-efficient-packet-sniffer/331041/10>

Ssaf. (n.d.). How raw sockets work? Retrieved from <https://superuser.com/questions/1678667/how-raw-sockets-work>

GeeksforGeeks. (2025, July 15). Kali Linux sniffing and spoofing. Retrieved from <https://www.geeksforgeeks.org/linux-unix/kali-linux-sniffing-and-spoofing/>

ChatGPT. (n.d.). Retrieved from <https://chatgpt.com/>